Learning Objectives

Learners will be able to...

- **Define a stack**
- **Cover Basic operations of stacks**
- **Perform Basic Operations with stacks**

Stacks

What are Stacks?

A **stack** is an abstract data type (ADT) that allows all data operations at one end only. At any given time, we can only access the top element of a stack. You cannot directly access elements elsewhere in the stack.

Stacks are also as a linear data structure, but it may be more helpful to think of the stack data structure as a physical stack of books. If you want to add a book to the stack, it goes on top. If you remove a book, you take it from the top. Moreover, the most recently added book will be the first to be removed.



Basic Operations

A stack has two fundamental operations: **push** and **pop**. Pushing adds an element (insertion) to the top of the stack, while popping removes (deletion) the top element from the stack. Again, both of these operations only work on the top of the stack. Insertions and deletions cannot happen elsewhere in the stack.

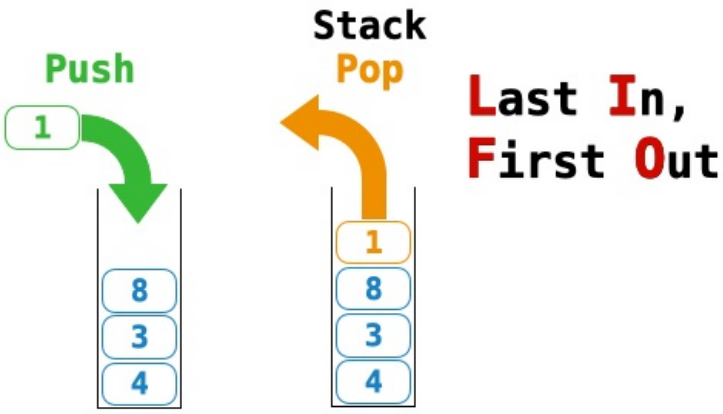
Another common operation used with stacks is **peek**. Peeking involves looking at the top element in the collection while keeping the element on the stack. Unlike pushing and popping, peeking is not a fundamental operation associated with the stack data structure. Stacks must be able to push and pop; peeking is optional.

You may have noticed that traversing is not an operation associated with a stack. Stacks cannot be directly traversed. If you want to interact with elements in the stack, you first must pop them off the data structure.

LIFO

The order in which elements come off and on a stack is always the same. The last element added is the first one to be deleted. This consistent behavior is referred to as last in, first out (LIFO). LIFO is pretty efficient

since we always know which element can be removed and where the next element will be added.



Now that we know what a stack is, we are going to see how to implement one in a couple of different ways.

Array-Based Stack Implementation

Representing a Stack with an Array

The stack abstract data type can be implemented using various underlying data structures, with arrays and linked lists being the most commonly used. The choice between these implementations often depends on the specific requirements of the application.

In an array-based stack, a variable, often named `top`, is used to keep track of the index of the most recently added element. This is essential for adhering to the last-in, first-out (LIFO) behavior intrinsic to stacks. The `top` variable allows for quick and efficient `push`, `pop`, and `peek` operations, which are the core functions of a stack.

Arrays are often the go-to data structure for implementing stacks due to their ability to access elements in constant time. However, a limitation of using arrays is their fixed size. One must either define the maximum size of the stack beforehand or use dynamic resizing techniques.

Key Operations and Their Time Complexities

Here are the essential operations for an array-based stack along with their time complexities:

- **Push**: $O(1)$
- **Pop**: $O(1)$
- **Peek**: $O(1)$
- **IsEmpty**: $O(1)$
- **IsFull**: $O(1)$

Implementation

Create the `ArrayStack` class. It should have three attributes: `maxSize`, `top`, and `stackArray`. Since we are using an array, we need to know the size of the array to be used. This is done with `maxSize`. The `top` attribute represents the top element in the stack. Finally, `stackArray` is the actual array used to store data in the stack.

```
class ArrayStack {
private:
    // Initialize the maximum size of the stack
    int maxSize;
    // Initialize a variable to track the top element
    int top;
```

```
    // Create an array to hold the stack elements
    int* stackArray;

public:
    // Constructor to set up the stack size
    ArrayStack(int size) {
        maxSize = size;
        stackArray = new int[maxSize];
        // Set the top to -1 as the stack is initially empty
        top = -1;
    }

    // Destructor to clean up the allocated memory
    ~ArrayStack() {
        delete[] stackArray;
    }

    // Push operation to add a value to the top of the stack
    void push(int value) {
        // Check if the stack is full
        if (isFull()) {
            std::cout << "Stack is full. Cannot push " << value
        << std::endl;
            return;
        }
        // Increment the top and add the value
        stackArray[++top] = value;
    }

    // Pop operation to remove the top value from the stack
    int pop() {
        // Check if the stack is empty
        if (isEmpty()) {
            std::cout << "Stack is empty. Cannot pop." <<
        std::endl;
            return -1;
        }
        // Remove the top value and decrement the top
        return stackArray[top--];
    }

    // Peek operation to view the top value without removing it
    int peek() {
        // Check if the stack is empty
        if (isEmpty()) {
            std::cout << "Stack is empty. Cannot peek." <<
        std::endl;
            return -1;
        }
        // Return the top value without removing it
        return stackArray[top];
    }

    // Check if the stack is empty
    bool isEmpty() {
        return (top == -1);
    }
```

```
    // Check if the stack is full
    bool isFull() {
        return (top == maxSize - 1);
    }
};
```

In the main function, instantiate an `ArrayStack` with a size of five elements. Push integers `1`, `2`, and `3` onto the stack. Peek at the top value and print it. Then pop the top element off the stack and print it. Finally, peek at the new top element and print it.

```
int main() {
    // Create a stack of size 5
    ArrayStack myStack(5);
    // Push elements onto the stack
    myStack.push(1);
    myStack.push(2);
    myStack.push(3);
    // Peek at the top element and display it
    std::cout << "Peek: " << myStack.peek() << std::endl;
    // Pop an element and display it
    std::cout << "Pop: " << myStack.pop() << std::endl;
    // Peek at the new top element and display it
    std::cout << "Peek: " << myStack.peek() << std::endl;

    return 0;
}
```

You should see the following output:

```
Peek: 3
Pop: 3
Peek: 2
```

Creating a stack with an array involves managing the `top` index to represent the top of the stack. Pushing and popping operations adjust this index accordingly, while peeking simply accesses the element at `top`.

List-Based Stack Implementation

Representing a Stack with a Linked List

After exploring array-based stacks, let's turn our attention to linked list-based implementations. Linked lists offer more flexibility than arrays since they can dynamically grow and shrink, overcoming the fixed size limitation of array-based stacks. In a linked list-based stack, the head node of the linked list serves as the top of the stack.

Key Operations and Their Time Complexities

Here are the essential operations for a linked list-based stack along with their time complexities:

- **Push**: $O(1)$
- **Pop**: $O(1)$
- **Peek**: $O(1)$
- **IsEmpty**: $O(1)$

Implementation

Since this implementation uses a singly linked list, we first need to create the `Node` class. Each node stores data and contains a pointer to the next node. The pointer should be `nullptr` when creating a `Node` object.

```
// Node class definition
class Node {
public:
    int data;
    Node* next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

Next, create the `LinkedStack` class. The head attribute keeps track of the head node in the list, representing the top of the stack. It should be initialized to `nullptr` to indicate that the list is initially empty.

```
// LinkedStack class definition
class LinkedStack {
private:
    // Initialize the head (top) of the linked list
    Node* head;

public:
    // Constructor to initialize an empty stack
    LinkedStack() {
        head = nullptr;
    }

    // Push operation
    void push(int value) {
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
    }

    // Pop operation
    int pop() {
        if (isEmpty()) {
            std::cout << "Stack is empty. Cannot pop." <<
        std::endl;
            return -1;
        }
        int poppedValue = head->data;
        Node* temp = head;
        head = head->next;
        delete temp;
        return poppedValue;
    }

    // Peek operation
    int peek() {
        if (isEmpty()) {
            std::cout << "Stack is empty. Cannot peek." <<
        std::endl;
            return -1;
        }
        return head->data;
    }

    // Check if stack is empty
    bool isEmpty() {
        return head == nullptr;
    }
};
```

In the `main` function, instantiate a `LinkedStack` object. Since linked lists are dynamic, there's no need to specify a size. Push integers `1`, `2`, and `3` onto the stack. Peek at the top value and print it. Then pop the top element off the stack and print it. Finally, peek at the new top element and print it.

```cpp
int main() {
    // Create a LinkedStack instance
    LinkedStack myStack;

    // Push elements onto the stack
    myStack.push(1);
    myStack.push(2);
    myStack.push(3);

    // Peek at the top element and display it
    std::cout << "Peek: " << myStack.peek() << std::endl;
    // Pop an element and display it
    std::cout << "Pop: " << myStack.pop() << std::endl;
    // Peek at the new top element and display it
    std::cout << "Peek: " << myStack.peek() << std::endl;

    return 0;
}
```

You should see the following output:

```
Peek: 3
Pop: 3
Peek: 2
```

Implementing a stack with a linked list involves managing nodes where each node stores data and a pointer to the next node. Operations such as `push`, `pop`, and `peek` are performed at the head of the list to maintain a time complexity of $O(1)$.

Array-Based vs Linked List-Based Stacks

Comparing Stack Implementations

Having explored both array-based and linked list-based stacks, it's essential to compare them to understand their strengths and weaknesses better. This will help you make a more informed choice depending on your specific needs.

We will compare the two implementations with regards to memory allocation, time complexity, ease of implementation, and use cases.

Memory Allocation

Array-based stacks require that memory be allocated beforehand. It is possible to create a new array of a different size and move over all of the elements if need be. However, these additional operations can be costly.

Linked list-based stacks do not need to worry about memory allocation beforehand as linked lists are dynamic. However, nodes in a linked list require a bit of extra memory for pointers.

Time Complexity

Array-based stacks have a set of three core operations: `push`, `pop`, and `peek`. Due to the direct access nature of arrays, these operations have a time complexity of $O(1)$. This performance assumes that an array does not need to be resized. Dynamically altering the array will cause additional overhead.

Linked list-based stacks have the same set of core operations. Because stacks only allow operations at the top of the stack, that means all operations happen at the head of the list. Because this position is always known, `push`, `pop`, and `peek` also have a time complexity of $O(1)$.

Ease of Implementation

Array-based stacks are a bit easier to implement and use as they use a built-in data structure. However, the fixed-size nature of arrays means you need to check if the stack is full before pushing.

Linked list-based stacks are slightly more complex due to creating the classes for the nodes and the linked list. In addition, you have to update the `head` attribute so that it always points to the top of the stack. Linked lists are dynamic, so you can push to the stack without checking to see if it is full.

Use Cases

Array-based stacks make the most sense when the size of the stack is known and does not change. This implementation also lends itself nicely when you want a quick and simple solution.

Linked list-based stacks make the most sense if the stack needs to frequently change size. Using a linked list for a stack offers more flexibility if you do not mind managing pointers.

In summary, array-based stacks are simpler but can have limitations with dynamic sizing. Linked list-based stacks are more flexible but a bit more complex to implement. Neither implementation is more performant than the other. The choice between the two will largely depend on your application's specific requirements for memory use and computational efficiency.

Formative Assessment 1

Formative Assessment 2